

B6: OAuth Scopes

Users grant applications access to permissioned data through OAuth, the same authorization layer the rest of atproto uses. The unit of grant is the **space:** scope. This lesson covers the scope grammar and its defaults, the two read grants (**read** and **read_self**), how a request gets matched against a grant, the **manage** verbs for administering spaces themselves, how scopes render on the consent screen, and how space permissions bundle into permission sets.

One idea runs under all of it: the space *type* is the consent boundary. From the "Space type" section of the proposal:

The type is also the **OAuth consent boundary**. Access is granted to a user by type, e.g. "access to your AtmoBoards forums" (see OAuth scopes).

Permissioned Data proposal, "Space type"

The **space:** scope grammar

A user grants an application access to spaces through a **space:** OAuth scope. The scope identifies a set of spaces by their (**authority**, **spaceType**, **skey**) identifier and states what the grant permits within them.

Permissioned Data proposal, "OAuth scopes"

The grammar, exactly as the proposal gives it:

`space:<spaceType>[?authority=<did>] [&skey=<skey>] [&collection=<nsid>...] [&action=<action>]`



The space type is positional and required; everything else is a query parameter. Three of the parameters may repeat (**collection**, **action**, **manage**). The full parameter table:

Parameter	Position	Multiple	Default	Values
spaceType	positional (required)	no	N/A	a space-type NSID, or * for any type
authority	query	no	self	a space authority DID, self for the granting user's own DID, or * for any authority

Parameter	Position	Multiple	Default	Values
skey	query	no	*	a space key (1–512 chars), or * for any key
collection	query	yes	the space type lexicons declared collections	a record collection NSID, or * for any
action	query	yes	read, create, update, delete	read_self, read, create, update, delete
manage	query	yes	(none)	create, update, delete

The parameters split into two jobs:

authority, **spaceType**, and **skey** select **which spaces** the grant covers, matching the first three segments of a space URI. **action** (and **collection**) govern operations on the **records** in those spaces. **manage** governs operations on the **spaces themselves**.

Permissioned Data proposal, "OAuth scopes"

The defaults, and why **authority=self** matters

The defaults are not uniform, and the differences are where mistakes happen. **authority** defaults to **self**, the granting user's own DID, not a wildcard. **skey** defaults to *, any space key. **collection** defaults to the **collections** array declared by the space type's declaration (more on this under Matching). **action** defaults to the full record-action set: **read, create, update, delete**. And **manage** defaults to nothing at all, so no grant carries administrative capability unless it asks for it.

authority defaults to **self**, the granting user's own DID, so a bare **space:<spaceType>** grant covers only the user's own spaces of that type. Reaching spaces under other authorities (e.g. a shared forum anchored on an app or another user) requires naming that authority, or **authority=*** for any.

Permissioned Data proposal, "OAuth scopes"

So a bare **space:com.example.bookmarks** is the typical grant for personal data: only the user's own bookmarks spaces, with read access plus write access to the collections the type's declaration lists. A forum client is the opposite case. Its spaces are anchored on other people's or an app's DID, so it has to ask for **authority=*** (or a specific authority DID) to reach them.

Read access: **read** vs **read_self**

There are two read verbs in **action**, and the line between them decides whether an app can see other members' data at all.

read: whole-space, all-or-nothing

read is all-or-nothing at the space boundary. There is no partial, per-record, or per-author whole-space read grant.

Permissioned Data proposal, "Read access"

A **read** grant confers two things:

- access to the read and sync methods (**com.atproto.space.getRecord**, **listRecords**, **getBlob**, **getLatestCommit**, **getRepo**, **listRepoOps**) on the holder's own PDS, sufficient to read the holder's own repo in the space
- access to **com.atproto.space.getDelegationToken** for that space, which an application exchanges for a space credential to read *any* repo in the space

The second item is the bridge into the credential flow from section B3. Holding **read** is what entitles an app to mint delegation tokens. Those get traded to the space authority for a space credential, and the credential unlocks every member's repo host.

read_self: own repo only

read_self is the narrower grant. It confers access to the same read and sync methods, but only for the holder's **own** repo in the space, and it does **not** grant **getDelegationToken**. An application holding only **read_self** can read its user's own records but cannot reach the rest of the space. **read** implies **read_self**.

Permissioned Data proposal, "Read access"

Because **read_self** withholds **getDelegationToken**, an app holding only **read_self** has no path to a space credential. It's not just barred from reading other members' repos directly; it can't enter the credential flow in the first place. That's the right grant for something like a personal export or backup utility, which should see the user's own contributions to a space but never other members' data.

The subsumption rule goes one way: **read** implies **read_self**. A full read grant satisfies any request a **read_self** grant would authorize.

Which credential authorizes which method

A **space credential** grants whole-space read/sync access directly, so the read and sync methods accept **either** a covering OAuth scope or a space credential. Write methods accept only an OAuth credential, since a write is attributed to the authoring user.

Permissioned Data proposal, "Read access"

That's the read/write asymmetry in one sentence. A syncer holding only a space credential, with no OAuth session, can call **getRepo**, **listRepoOps**, **getRecord**, **listRecords**, **getBlob**, and

`getLatestCommit` against any member's repo host. But `createRecord`, `putRecord`, `deleteRecord`, and `applyWrites` demand OAuth, because every write is attributed to a specific authoring user and must ride a session delegated by that user. A space credential is also the *output* of the credential flow, not an input to it: `getDelegationToken` itself is an OAuth-authenticated PDS method.

Matching rules

A server authorizing a request checks it against the grant like this:

A request is authorized by a grant when its target space matches the grant's (`authority`, `spaceType`, `skey`) (each component equal to the grant's value or covered by its `*`) and the grant permits the requested operation, per the rules below. An `authority` of `self` matches only the space whose authority is the granting user's own DID and is resolved to that DID at grant time.

Permissioned Data proposal, "Matching"

So matching is two checks: the target space matches the grant's space selector component by component, with `*` covering anything, and the operation is permitted. For record operations, permission depends on the `action` verb:

- `read` covers every repo in the space and ignores `collection` since read access is all-or-nothing.
- `read_self` covers only the holder's own repo and is constrained by `collection`. A `read` grant also satisfies a `read_self` request.
- `create`, `update`, and `delete` act on a specific record, so they are additionally constrained by `collection`.

Two consequences follow. First, adding `collection=...` to a scope with `action=read` does not narrow what can be read. `read` pays no attention to collection, so every record in every repo of the space stays readable; collection constraints bite only on writes and on `read_self`. Second, if you want `read_self` to cover *all* of the user's own records rather than just the declared ones, you have to say `collection=*` explicitly.

Omitting `action`

Omitting `action` grants `read`, `create`, `update`, and `delete` (`read` is inclusive of `read_self`).

Permissioned Data proposal, "Matching"

Conversely, an *explicit* `action` list replaces the default entirely. A scope that says `action=read_self` grants only `read_self`: no writes, no whole-space read.

Omitting **collection**: the declaration default and dynamic updates

Omitting **collection** defaults it to the **collections** declared by the space type's declaration. A bare **space:com.atmoboards.forum** grant therefore permits writing the forum's own record types, the same way a bare **repo:** scope permits writing the collections it names. An application may narrow this by listing a subset of collections, or widen it with **collection=***. When **spaceType** is ***** there is no declaration to draw from, so the default is empty and the grant confers no write targets unless provided.

Permissioned Data proposal, "Matching"

Recall from B2 that a space type declaration carries a **collections** array: the collections clients should expect in a space of that type. That array is exactly what a bare scope's **collection** defaults to. The wildcard-type case behaves differently: **space:*?authority=did:plc:abc123** reads every space under that authority, of any type, but since there is no single declaration to supply a collection default, it confers *no write targets at all* unless **collection** is spelled out.

The default is also *live*, not a snapshot:

This default follows the same dynamic-update semantics as a permission set. It is resolved from the declaration as it stands when the grant is evaluated, not frozen at consent time. If the space type's declaration later adds a collection to **collections**, existing bare grants widen to include it once the change propagates, subject to the caching/rollout window of the authorization server that resolves the declaration. An application that does not want its authorized collection set to move with the declaration should enumerate collections explicitly rather than rely on the default.

Permissioned Data proposal, "Matching"

So a bare grant's write surface tracks the declaration over time, with no re-consent and no re-request. If the forum type later declares **com.atmoboards.poll**, every existing bare forum grant gains poll-writing once the change propagates. Enumerating collections explicitly is the opt-out.

Manage verbs

manage is the third axis, entirely separate from **action**:

Space management operations are governed by **manage**, which takes the same **create/update/delete** verbs applied to the space itself rather than to its records. **manage** ignores **collection**. It is omitted by default, so an ordinary record-access grant confers no administrative capability.

Permissioned Data proposal, "Matching"

The verbs are the same words (**create**, **update**, **delete**) but the object is different: the space, not its records. There is no **action=manage**; **manage** is its own parameter. **collection** is irrelevant to it, since a space is not a collection of a particular record type. And because the default is empty, a normal record-access scope carries zero administrative power.

What each verb concretely permits is not fixed by the protocol:

The protocol does not enumerate what each **manage** verb permits, because space management is implementation-defined (see **simplespace**). Each space-management implementation maps the verbs onto its own administrative surface. For example, in **com.atproto.simplespace**, **manage=update** authorizes **com.atproto.simplespace.updateSpace** as well as **addMember** and **removeMember**.

Permissioned Data proposal, "Matching"

manage=create and **key=***

manage=create authorizes creating a space of the given **spaceType** under the given authority. Unlike every other operation, it concerns a space that does not yet exist, so scoping it to a concrete **key** is unusual. It is typically granted with **key=*** ("this app may create spaces of this type").

Permissioned Data proposal, "Matching"

The reasoning is specific to **create**. **manage=update** and **manage=delete** act on spaces that already exist and can sensibly be pinned to one **key**, but a space being created has no key to pin to yet.

Worked examples

The proposal's own examples, verbatim. Try to predict each scope string's meaning before reading its gloss; that prediction is the skill the grammar exists to exercise.

- **space:com.example.bookmarks**: the user's own bookmarks space, with read access and write access to the collections its declaration lists. **authority** defaults to **self**, so this is the typical grant for personal data.
- **space:com.atproto.boarding.passenger?authority=***: every **com.atproto.boarding.passenger** space the user is in under any authority, with read access for the entire space and write access to the collections the forum's declaration lists (**com.atproto.boarding.thread**, **com.atproto.boarding.reply**). This is the typical grant for a forum client, which reads forums hosted by others.
- **space:com.atproto.boarding.passenger?authority=*&action=read**: the same spaces, read-only. No **collection** is needed because **read** is not constrained by collection.
- **space:com.atproto.boarding.passenger?authority=*&action=read_self&collection=***: read-only, and only the user's own repo in those forums. Suitable for a personal export or backup tool that

should not see other members' posts. The `collection=*` is required to read all records, not just those declared in the Lexicon.

- `space:com.atmoboards.forum?authority=*&collection=*`: read access plus write access to every collection, not only the declared ones.
- `space:com.atmoboards.forum?authority=did:plc:abc123&skey=default&collection=com.atmoboards.thread&action=create&action=update`: create and update `com.atmoboards.thread` records in the forum keyed `default` under authority `did:plc:abc123`.
- `space:com.atmoboards.forum?authority=*&action=read_self&manage=update&manage=delete`: administer the user's forums (update and delete the spaces), with read access, but no record-write access.
- `space:com.atmoboards.forum?authority=*&manage=update&manage=delete`: administer the user's forums (update and delete the spaces), with full read/write access to records in the space.
- `space:?*authority=did:plc:abc123`: read every space under authority `did:plc:abc123`, any type.

The last two `manage` examples are worth comparing. Both administer the same spaces, but the record-level access differs entirely: one pins `action=read_self` (own-repo reads only, no writes) while the other omits `action` and so receives the full default set (`read`, `create`, `update`, `delete`). `manage` and `action` compose independently.

Consent

Because scopes are shown to users, every element of a `space:` scope needs a user-legible rendering.

A `space:` scope is presented to the user on the OAuth consent screen and requires user-legible text. Each space `type` resolves to a space type declaration. The consent screen then displays the declaration's `name` (e.g. "AtmoBoards Forum") in place of the raw NSID.

Permissioned Data proposal, "Consent"

That `name` field (required, 1–64 characters, with optional `name:lang` localizations) exists in the declaration precisely for this screen. The authority gets its own presentation rule:

If a particular `authority` DID is specified in a scope, it should be presented to the user as the bidirectionally linked handle associated with the DID. If no handle bidirectionally validates, then the DID itself should be shown. An `authority` of `self` refers to the user's own account and needs no such presentation.

Permissioned Data proposal, "Consent"

So the handle comes first, and only if it bidirectionally validates against the DID. The raw DID is the fallback. `self` gets no presentation at all, since that is the user's own account. The broadest possible request gets special treatment:

A scope may request wildcard access on both `authority` and `spaceType`. This is a very broad grant, and as such the consent screen should present such a scope with a prominent warning.

Permissioned Data proposal, "Consent"

Permission sets

Applications often want to request several related grants under one friendly label. Space permissions plug into atproto's existing permission-set mechanism:

Space permissions can also be bundled, usually with more user-friendly verbiage, into a permission set.

Permissioned Data proposal, "Permission sets"

The proposal's example set:

```
{
  "type": "permission-set",
  "title": "AtmoBoards",
  "detail": "Read and post in your AtmoBoards forums",
  "permissions": [
    {
      "type": "permission",
      "resource": "space",
      "spaceType": "com.atmboards.forum",
      "authority": "*",
      "collection": ["com.atmboards.thread", "com.atmboards.reply"],
      "action": ["read", "create"]
    },
  ]
}
```

Within a permission, `"type": "permission"` is the entry discriminator, `"resource": "space"` selects the space resource, and the remaining fields carry the same parameters as the `space:` scope string: `spaceType`, `authority`, `skey`, `collection`, and `action`.

One hard constraint distinguishes a set permission from a standalone scope:

A set permission must name a concrete space type. In other words, the `spaceType` parameter may **not** be a wildcard inside a permission set. The other parameters may still be wildcards,

including **authority** and **collection**. A cross-type grant (**spaceType=***) is expressible only as a standalone **space:** scope requested directly.

Permissioned Data proposal, "Permission sets"

The example set itself uses **"authority": "*"**: wildcards are fine everywhere except **spaceType**. There is also a namespace rule, and it is asymmetric:

When expressing space resources in a permission set, the **spaceType** must follow the Namespace Authority requirements associated with permission sets. However, the collection parameter may be a wildcard or list collections under a different namespace authority than the space and the permission set.

Permissioned Data proposal, "Permission sets"

The **spaceType** is namespace-bound to the set's authority. The **collection** list is not; it may wildcard or reach into other namespaces.

You should now be able to explain

- The full **space:** scope grammar: the positional **spaceType** and the **authority**, **skey**, **collection**, **action**, and **manage** query parameters, including which may repeat.
- Every parameter's default: **authority=self**, **skey=***, **collection** from the space type declaration, **action** covering all four record verbs, and **manage** empty.
- What a bare grant like **space:com.example.bookmarks** covers, and why reaching spaces under other authorities requires **authority=<did>** or **authority=***.
- Which spaces and operations a grant's (**authority**, **spaceType**, **skey**) selector matches, and that **self** resolves to the granting user's DID at grant time.
- The two things a **read** grant confers, including access to **getDelegationToken**.
- How **read_self** differs: own repo only, constrained by **collection**, no **getDelegationToken**, and that **read** implies **read_self**.
- Why **read** is all-or-nothing at the space boundary and ignores **collection**, while **create/update/delete** (and **read_self**) are collection-constrained.
- Which methods accept a space credential versus an OAuth scope, and why writes are OAuth-only.
- The dynamic-update semantics of the **collection** default, and how an app opts out by enumerating collections explicitly.
- Why **spaceType=*** yields an empty collection default and therefore no write targets unless **collection** is supplied.
- What the **manage** verbs govern, that they ignore **collection**, that no administrative capability is granted by default, and why **manage=create** is typically paired with **skey=***.
- How the consent screen renders a space type (the declaration's **name**), a concrete authority DID (bidirectionally linked handle, DID fallback), **self**, and a double-wildcard scope (prominent

warning).

- How a space permission is expressed inside a permission set, which parameter may not be a wildcard there, and the namespace-authority rules for `spaceType` versus `collection`.